Google Skills

Two ways to define agents

In modules 1 to 3, you've been creating agents using **Python code** in `agent.py`:

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Helps students learn algebra',
    instruction='You are a patient math tutor...'
)
```

This works great for development. But ADK also supports defining agents using **YAML configuration files,** which can be easier for:

- Non-programmers to edit
- Version control and sharing
- Quick experimentation
- Separating configuration from code

This module introduces **Agent Config**, which is ADK's YAML-based approach to building agents.

Language support note

Important: ADK supports multiple programming languages:

- **Python:** Full support for both Python code and YAML Agent Config
- **Java:** Supports Python code-based agents (see Java Quickstart)

Agent Config (YAML) is currently Python-only. From the ADK docs:

"**Programming language:** The Agent Config feature currently supports only Python code for tools and other functionality requiring programming code."

This module focuses on the YAML Agent Config feature, which is available when using ADK with Python.

2 methods to define agents

Reference: ADK docs – Agent Config

Method 1: Python code (modules 1-3)

Created with: `adk create my_agent`

Generates:

```
None
my_agent/
├── agent.py      # Python code defining your agent
├── __init__.py
└── .env
```

agent.py:

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='assistant_agent',
    description='A helpful assistant',
    instruction='You are a helpful assistant.'
)
```

Method 2:
YAML configuration (this module)

Created with:

```
adk create --type=config my_agent
```

Generates:

```
None
my_agent/
├── root_agent.yaml   # YAML
configuration file
└── .env
```

root_agent.yaml:

```
None
name: assistant_agent
model: gemini-2.5-flash
description: A helpful assistant
instruction: You are a helpful
assistant.
```

From ADK docs:

"The ADK Agent Config feature lets you build an ADK workflow without writing code. An Agent Config uses a YAML format text file with a brief description of the agent."

Creating a YAML-based agent

Reference: ADK docs – Agent Config: Build an Agent

Now, let's create your first YAML-based agent step by step. The process is similar to what you learned in module 1, but with a key difference: Instead of generating Python code, we'll generate a YAML configuration file.

Step 1: Create the agent project

Run the following command to create a config-based agent:

```
Shell
adk create --type=config
my_config_agent
```

Notice the `--type=config` flag. This tells ADK to create a YAML-based agent instead of the default Python code-based agent.

From ADK docs:

"This command generates a `my_agent/` folder, containing a `root_agent.yaml` file and an `.env` file."

You'll see a new folder `my_config_agent/` with:

- `root_agent.yaml` – Your agent configuration (instead of agent.py)
- `.env` – Environment variables for API keys (same as module 1)

Step 2: Configure your API key

Just like in module 1, you need to configure your .env file with your API key. The process is identical: Open `my_config_agent/.env`, and add your `GOOGLE_API_KEY` as you did in module 1, step 7.

If you need a refresher, refer back to Module 1 – Step 7: Configure your API key.

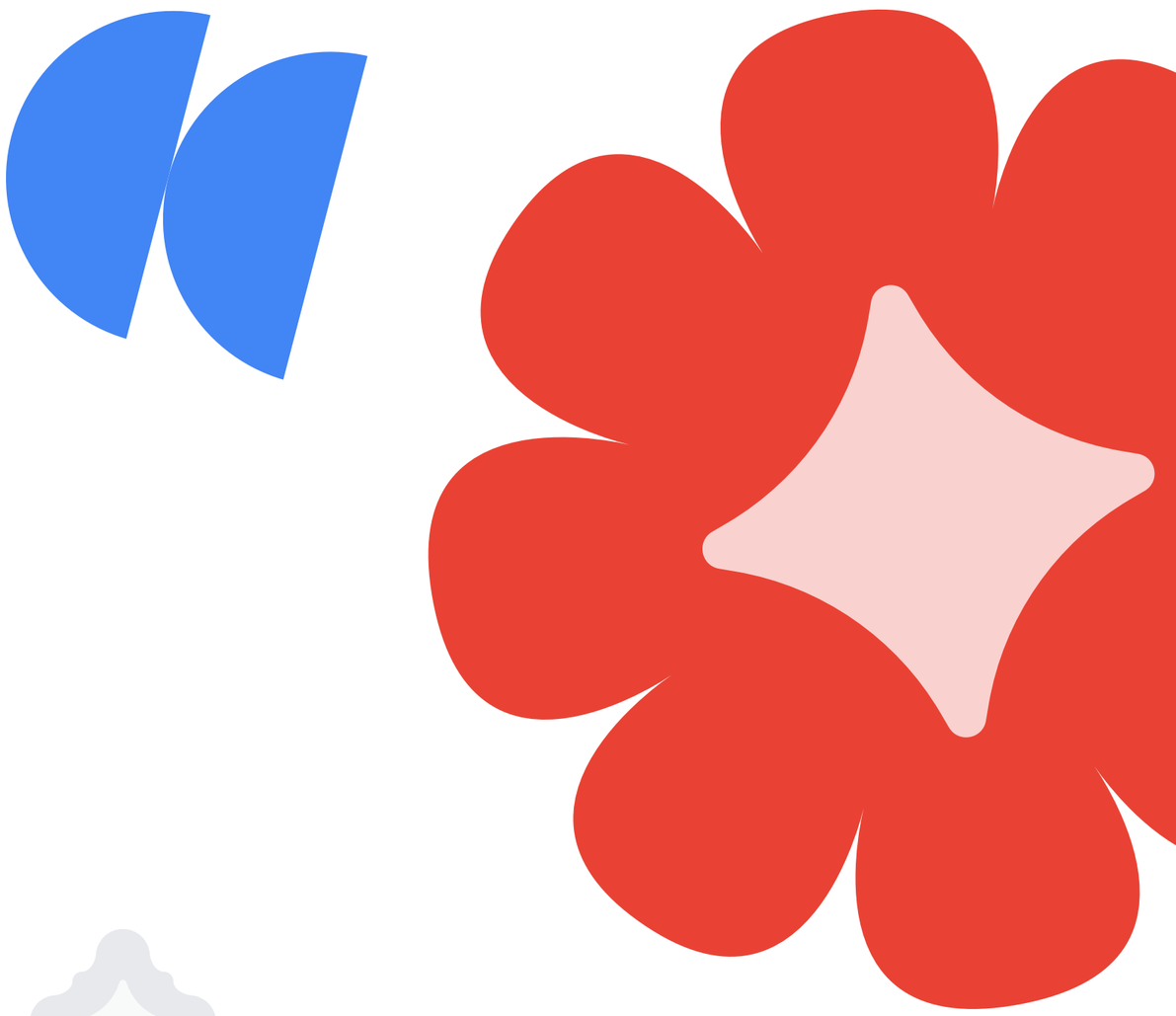
Step 3: Edit the YAML configuration

This is where things get interesting. Instead of writing Python code, you'll edit a YAML file.

Open `root_agent.yaml` in your text editor. The default generated file looks like this:

```
None
# yaml-language-server: $schema=https://raw.githubusercontent.com/google/adk-python/
refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
name: assistant_agent
model: gemini-2.5-flash
description: A helper agent that can answer users' questions.
instruction: You are an agent to help answer users' various questions.
```

This is a basic agent configuration with four key parameters.
Notice how clean and readable YAML is compared to Python code.
There are no imports, no function definitions, just straightforward key-value pairs.



Let’s customize this to create the same math tutor agent you built in module 2.
Edit your `root_agent.yaml` file to look like this:

```
None
# yaml-language-server: $schema=https://raw.githubusercontent.com/google/adk-python/
refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
name: math_tutor_agent
model: gemini-2.5-flash
description: Helps students learn algebra by guiding them through problem-solving
steps.
instruction: |
  You are a patient and encouraging algebra tutor.

  Your teaching approach:
  1. When a student asks a question, first understand what they're struggling with
  2. Break down the problem into smaller, manageable steps
  3. Guide them to discover the answer rather than giving it directly
  4. Provide positive reinforcement for effort and progress
  5. Use simple language and avoid jargon

  Always maintain a supportive, patient tone. Learning takes time, and every
  question is an opportunity to grow.
```

Understanding YAML syntax:

The vertical bar `|` after `instruction:` tells YAML everything that follows is multi-line text. This is perfect for writing detailed instructions that span multiple lines. You typically don't need quotes or special escaping. Just write your instructions naturally.

Each parameter maps directly to what you learned in module 2:

- `name`: The agent’s unique identifier
- `model`: Which LLM to use (`gemini-2.5-flash`)
- `description`: What the agent does (for other agents to understand)
- `instruction`: How the agent should behave (the most important part)

Step 4: Run your YAML-based agent

Here's the great part: **Running a YAML-based agent is exactly the same as running a Python-based agent.** All the commands you learned in module 3 work identically.

From your workspace directory (the parent folder containing `my_config_agent`), run:

```
Shell
adk web my_config_agent
```

This works the same way as it did in modules 1 and 2. ADK loads your `root_agent.yaml` file, creates the agent based on that configuration, and starts the web interface.

From ADK docs:

"You can run your Agent Config-defined agent... `adk web` - Run web UI interface for your agent."

Once the web interface opens, interact with your math tutor agent just like you did in module 2. Try asking: "How do I solve 2x + 5 = 13?" and watch it guide you through the problem step-by-step.

YAML versus Python: When to use each

Now that you've seen both approaches, you might be wondering "When should I use YAML configuration versus Python code?" Both methods create agents that work identically, but they're suited for different scenarios.

Aspect	YAML config	Python code
Ease of editing	✅ Very easy (text file)	⚠️ Requires coding knowledge
Best for	Simple agents, quick prototypes	Complex agents, custom logic
Version control	✅ Clean differences	⚠️ Code changes
Flexibility	⚠️ Limited to config options	✅ Full programmatic control
Non-programmers	✅ Can edit	❌ Need coding skills
Advanced features	⚠️ Coming soon	✅ Full access

Choose YAML when:

You want simplicity and accessibility. YAML configuration is perfect when you're building straightforward agents and want to keep things simple. Non-programmers can easily edit a YAML file to adjust agent behavior without touching any code—great for collaboration across teams where not everyone has coding experience.

You need quick experimentation. Do you want to test different instructions or switch between models? Just edit the YAML file, and restart your agent. There is no need to navigate through Python imports and class definitions.

Configuration should be separate from code. YAML keeps your agent's configuration cleanly separated from any custom logic you might add later. This makes it easier to manage different configurations for development, staging, and production environments.

Choose Python when:

You're building complex multi-agent systems. As you'll learn in course 7, sophisticated multi-agent architectures with delegation, workflow agents, and custom orchestration require the full power of Python code. YAML is limited to basic configuration options.

You need custom tools and callbacks. Courses 4 and 5 will teach you about adding custom tools and implementing callbacks. These advanced features require Python code and can't be fully expressed in YAML alone.

You want programmatic control. Sometimes you need to dynamically generate agent configurations, modify behavior based on runtime conditions, or integrate tightly with other Python libraries. Python code gives you this flexibility.

Example: Side-by-side comparison

To really understand the equivalence between YAML and Python, let's look at the same simple agent defined both ways. This greeting agent has the same four parameters we've been using throughout this course.

Python code (agent.py):

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='greeting_agent',
    description='A friendly greeting agent',
    instruction='Greet users warmly and professionally.'
)
```

YAML config (root_agent.yaml):

```yaml
name: greeting_agent
model: gemini-2.5-flash
description: A friendly greeting agent
instruction: Greet users warmly and professionally.
```

The result: Both produce the exact same agent behavior. When you run either version with `adk web`, you'll get an identical greeting agent. The YAML version is just cleaner and easier to read—no imports, no parentheses, no quotes (in most cases), just straightforward configuration.

What you've accomplished

Congratulations, you've now learned that there are **two ways to define agents in ADK**, and both are equally valid approaches depending on your needs.

You started with Python code in modules 1 to 3, where you created agents by writing `agent.py` files with imports and class instantiation. This gave you hands-on experience with the fundamental structure of agents.

Now, you've discovered YAML configuration, a simpler, code-free way to define agents. You learned that `adk create --type=config` generates a `root_agent.yaml` file instead of Python code, making agent creation accessible to non-programmers.

Most importantly, you understand that both approaches produce identical agents. Whether you define your math tutor in Python or YAML, it behaves exactly the same way. The same `adk web`, `adk run`, and `adk api_server` commands work for both.

You also learned when to choose each method—YAML for simplicity and quick prototyping and Python for complex multi-agent systems and advanced features you'll learn about in future courses.

Finally, you're aware that **Agent Config is currently Python-only**, although ADK itself supports multiple programming languages, including Java. As the framework evolves, YAML support may expand to other languages.

Key takeaways

ADK gives you flexibility in how you define agents. You don't have to write code if you don't want to. YAML configuration provides a clean, accessible alternative for simple agents.

YAML configuration shines when you need quick experimentation, when non-technical team members need to adjust agent behavior, or when you want a clean separation between configuration and code. You can edit a YAML file, save it, and restart your agent with zero programming.

Python code becomes essential as your agents grow more sophisticated. When you start adding custom tools (course 4), implementing callbacks (course 5), or building multi-agent systems (course 7), you'll need the full power of Python. But you can always start with YAML and migrate to Python when needed.

Both methods are first-class citizens in ADK. There's no "right" or "wrong" choice. Pick the approach that best fits your current needs. And remember, the ADK team is actively developing the Agent Config feature and welcomes feedback.

From ADK docs:

"The Agent Config feature is experimental and has some known limitations. We welcome your feedback!"